

The Java Game Development Tutorial

Graphics with JAVA

A computer screen is divided into rows and columns of pixels. Each pixel can be set to a different color. In order to be able to draw pictures on the screen, we need to be able to identify which pixel is which. The way to do this is with a coordinate system, which each pixel is identified with two numbers: the number of pixels from the left edge and the number of pixels down from the top. With a few simple commands we should be able to draw almost any picture we want. The commands are:

- Draw a line between two pixels
- Draw a rectangle
- Draw a rectangle and fill it with a specified color
- Draw a circle, or part of a circle
- Draw an oval
- Draw and fill polygon with an arbitrary number of sides
- Draw text on the screen at a particular location.

Here is a list of JAVA commands. All of these commands come from the **Graphics** class and to use them we must always refer to an object of this class which we will call **g**.

g.drawLine (x_1 , y_1 , x_2 , y_2)

//This draws a line from the point with pixel coordinates x_1 , y_1 to x_2 , y_2 .

g.drawRect (left_corner_x_coordinate, upper_corner_y_coordinate, with, height)

g.fillRect (left_corner_x_coordinate, upper_corner_y_coordinate, with, height)

g.drawOval (left_corner_x_coordinate, upper_corner_y_coordinate, with, height)

g.fillOval (left_corner_x_coordinate, upper_corner_y_coordinate, with, height)

g.drawArc (left_corner_x_coordinate, upper_corner_y_coordinate, with, height, starting_point_in degrees, size or arc in degrees)

g.fillArc (left_corner_x_coordinate, upper_corner_y_coordinate, with, height, starting_point_in degrees, size or arc in degrees)

You can also draw and fill a polygon by giving the coordinates of all the vertices. The first step is to create an object of class **Polygon** with a statement like this:

Polygon polygon_name = new **Polygon** (x_array , y_array , numpoints);

Here, x_array is the array of x coordinates of the vertices, y_array is the array of y coordinates of the vertices and numpoints is the number of vertices.

```
g.drawPolygon (poylgon_name);
```

```
g.fillPolygon (polygon_name);
```

Now I will give you five important reminders:

1. All distances are measured in pixels
2. Horizontal x coordinates are given before vertical y coordinates.
3. Vertical y coordinates are measured down
4. Make sure you follow all capitalization patterns
5. For each color we will use **setColor**. This is still to come!

Basic structure of an applet

Well, this is very simple and the basic structure of an applet will be the same for every applet you will program. The major difference between an applet and an application is that there is no public static void main (String[]args) - method in an applet! Instead every applet should implement the following methods:

```
//import necessary packages
import java.applet.*;
import java.awt.*;
```

```
// Inherit the applet class from the class Applet
public class FirstApplet extends Applet
{
```

```
    // Now you should implement the following methods
```

```
    // init - method is called the first time you enter the HTML site with the applet
    public void init() {... }
```

```
    // start - method is called every time you enter the HTML - site with the applet
    public void start() {... }
```

```
    // stop - method is called if you leave the site with the applet
    public void stop() {... }
```

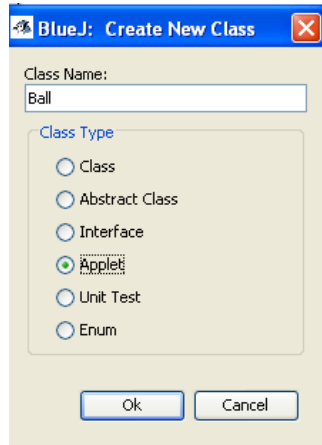
```
    // destroy method is called if you leave the page finally (e. g. closing browser)
    public void destroy() {... }
```

```
    /** paint - method allows you to paint into your applet. This method is called e.g. if you move your
    browser window or if you call repaint() */
    public void paint (Graphics g) { }
```

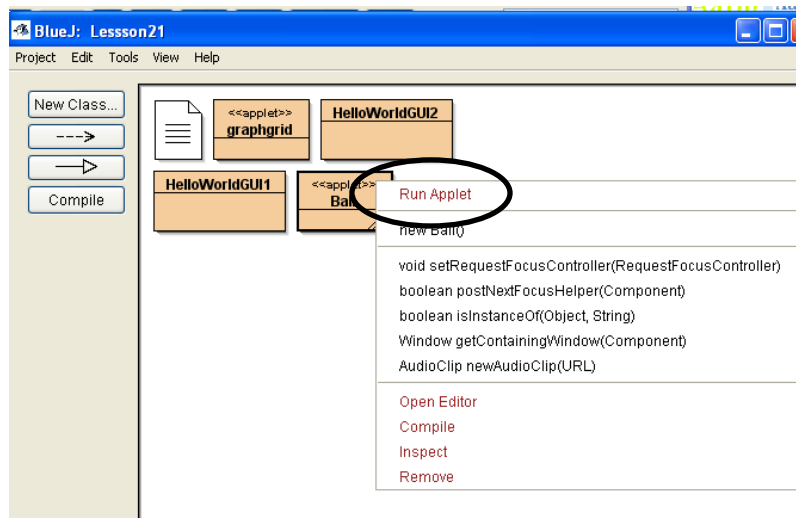
```
}
```

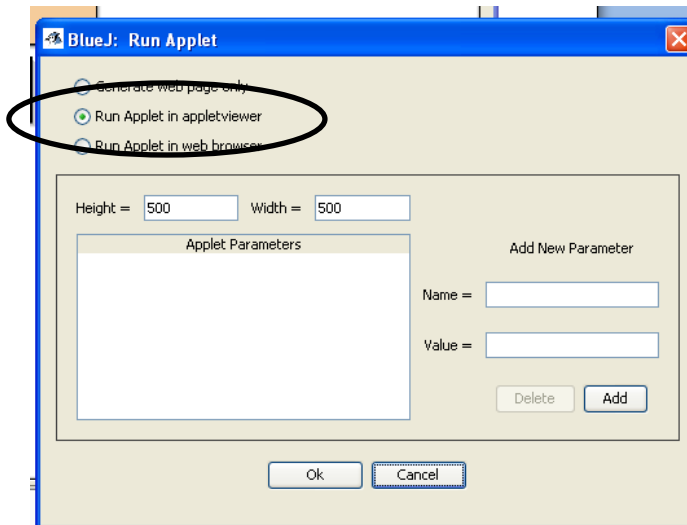
Using BlueJ to make Applets

BlueJ makes it simple to make applets. When you create a new class just select the applet button



BlueJ will automatically fill in appropriate code to create an applet. You should take some time and look at the code, but for now we will simply delete all the code and paste in the code that I have written. When you run your applet you will right click and select the RunApplet command that will run the Applet in the “viewer.”





How to move a ball

We want to start with a very essential step. We will program an applet in which a ball is moving from the left to the right hand side. I know this is nothing BIG but if you want to learn how to program games it is maybe the most important thing to understand how to animate objects!

At the beginning we have to write our basic structure of an applet again but we will add two little things. Our applet has to implement the interface `Runnable` and the corresponding method `run()` to animate a object. The structure of the applet should look like this:

```
import java.applet.*;
import java.awt.*;

public class Ball extends Applet implements Runnable
{

    public void init() { }

    public void start() { }

    public void stop() { }

    public void destroy() { }

    public void run () { }

    public void paint (Graphics g) { }
```

```
}
```

To move a object we need another object that has to be a instance of the class Thread, we declare this object in the start - method of our applet:

Threads

A thread is a piece of program that is able to run parallel to other parts of the program (multithreading). Threads are implemented by the class Thread, the interface Runnable and the method run(), we have already implemented these two things in the step before. Important methods of the class Thread are:

- Thread.start(): starts a thread
- Thread.stop(): stops a thread
- Thread.sleep(time in milliseconds): stops thread for a certain amount of time

You can find more functions of the thread class in the Java API!

And here comes the code!

```
public void start ()
{
    // define a new thread
    Thread th = new Thread (this);
    // start this thread
    th.start ();
}
```

Now this thread is running in the run() - method of our applet. Every time all methods... in the run - method have been called, we stop the thread for a short time. Your run method should look like this:

```
public void run ()
{
    // lower ThreadPriority
    Thread.currentThread().setPriority(Thread.MIN_PRIORITY);

    // run a long while (true) this means in our case "always"
    while (true)
    {
        // repaint the applet
        repaint();
    }
}
```

```

        try
        {

            // Stop thread for 20 milliseconds
            Thread.sleep (20);

        }
        catch (InterruptedException ex)
        {

            // do nothing

        }

        // set ThreadPriority to maximum value
        Thread.currentThread().setPriority(Thread.MAX_PRIORITY);

    }

}

```

What we have now is a never ending loop that executes all things within the loop, waits 20 milliseconds and executes everything once again and so on. But how can we move a circle that is painted by the applet?

Well this is a very simple idea: Our circle has a x - and a y - position. If we would add 1 to the x - position every time the thread executed, the ball should be moving across the applet, because it is painted at a different x - position every time we execute the thread!

Ok, let's start with drawing a circle: Add these lines to the paint - method of the applet:

```

public void paint (Graphics g)
{

    // set color
    g.setColor (Color.red);

    // paint a filled colored circle
    g.fillOval (x_pos - radius, y_pos - radius, 2 * radius, 2 * radius);

}

```

And we need the following instance variables at the head of the program:

```

int x_pos = 10;
int y_pos = 100;
int radius = 20;

```

To move a ball we change the value of the x_pos variable every time the thread is executed. Our run - method should look like this:

```
public void run ()
{
...

    while (true)
    {

        // changing the x - position of the ball/circle
        x_pos ++;
        ...

    }

}
```

Here is the full program Copy and paste and compile this program.

For homework, change the ball to another shape and make that shape move vertically – up and down in the center of the screen.

```
import java.applet.*;
import java.awt.*;

public class Ball extends Applet implements Runnable
{

    int x_pos = 10;        // x - Position of ball
    int y_pos = 100;       // y - Position of ball
    int radius = 20;       // Radius of ball

    public void init()
    {

        setBackground (Color.blue);

    }

    public void start ()
    {

        // Create a new thread in the running game
```

```

        Thread th = new Thread (this);

        // Start the thread

        th.start ();

    }

    public void stop()

    {

    }

    public void destroy()

    {

    }

    public void run ()

    {

        // Lower ThreadPriority draw to facilitate

        Thread.currentThread().setPriority(Thread.MIN_PRIORITY);

        // is true as long as the thread to run again

        while (true)

        {

            // Changing the x-coordinate

            x_pos ++;


            // Redraw the applet

            repaint();

            try

            {

                // Stop thread for specified milliseconds in parentheses

                Thread.sleep (20);

```

```
    }  
    catch (InterruptedException ex)  
    {  
        // do nothing  
    }  
    // Reset the thread to maximum value  
    Thread.currentThread().setPriority(Thread.MAX_PRIORITY);  
}  
}  
public void paint (Graphics g)  
{  
    g.setColor (Color.red);  
    g.fillOval (x_pos - radius, y_pos - radius, 2 * radius, 2 * radius);  
}  
}
```